

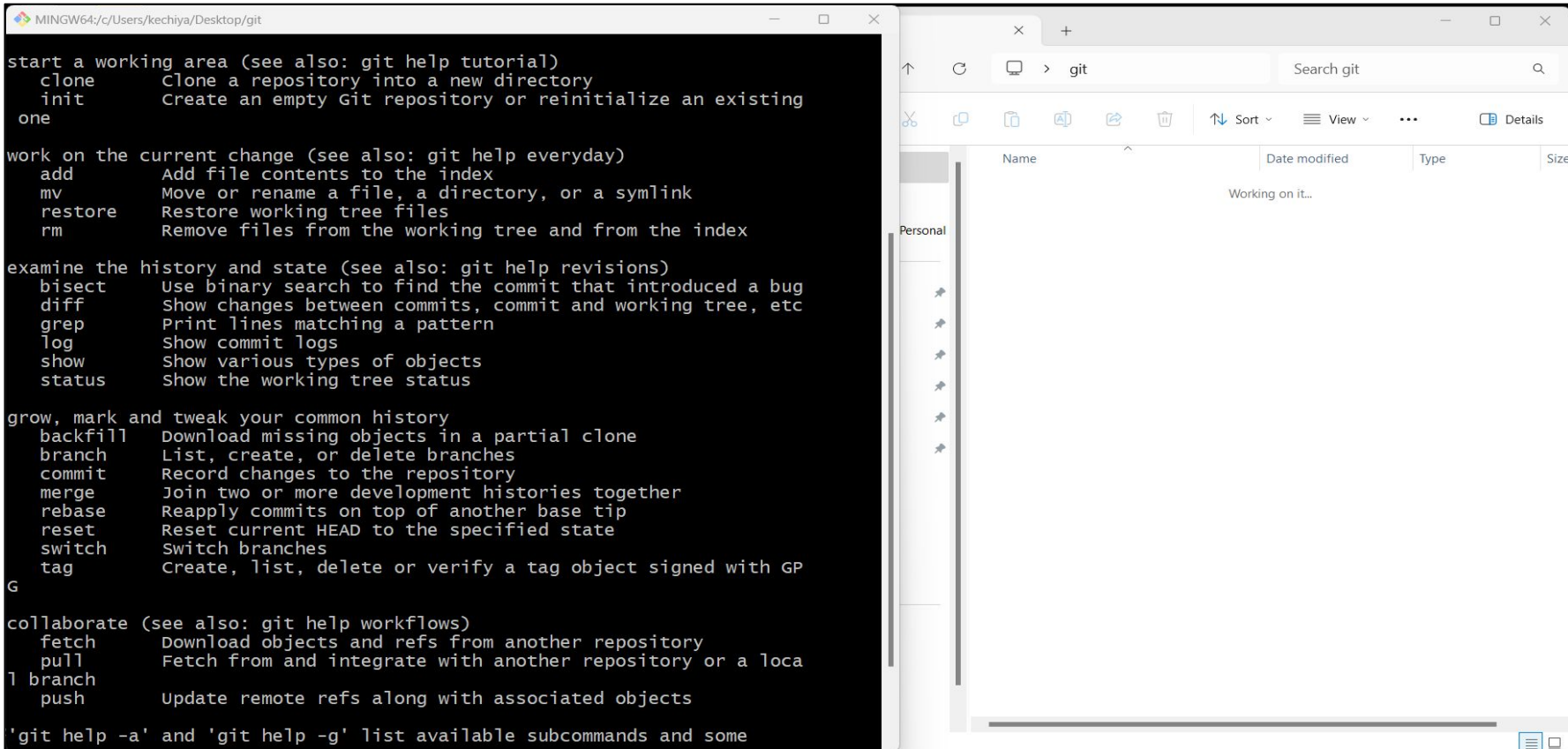
GIT

Git is a distributed version control system that helps track changes in files, manage code versions, and enable collaboration among developers efficiently.

Key Features of Git:

1. **Version Control:** Tracks changes to files, allowing you to revisit or revert to earlier versions if needed.
2. **Distributed System:** Each user has a complete copy of the project history, enabling offline work.
3. **Branching and Merging:** Supports creating separate "branches" for new features or experiments, which can later be merged into the main codebase.
4. **Collaboration:** Facilitates teamwork by handling conflicts and synchronizing changes from multiple contributors.
5. **Performance:** Optimized for speed and efficiency, even for large projects.

Installed git in my local machine



The image shows a Windows desktop with two windows. The left window is a terminal titled 'MINGW64/c:/Users/kechiya/Desktop/git' displaying the output of 'git help'. The right window is a file explorer titled 'git' showing the contents of the local git directory.

Terminal Output:

```
start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one
work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index
examine the history and state (see also: git help revisions)
  bisect    Use binary search to find the commit that introduced a bug
  diff      Show changes between commits, commit and working tree, etc
  grep      Print lines matching a pattern
  log       Show commit logs
  show      Show various types of objects
  status    Show the working tree status
grow, mark and tweak your common history
  backfill  Download missing objects in a partial clone
  branch    List, create, or delete branches
  commit    Record changes to the repository
  merge     Join two or more development histories together
  rebase    Reapply commits on top of another base tip
  reset     Reset current HEAD to the specified state
  switch    Switch branches
  tag       Create, list, delete or verify a tag object signed with GP
G
collaborate (see also: git help workflows)
  fetch     Download objects and refs from another repository
  pull      Fetch from and integrate with another repository or a local branch
  push      Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
```

File Explorer: The file explorer shows the 'git' directory. The left sidebar has a 'Personal' section with several star icons. The main pane shows a table with columns 'Name', 'Date modified', 'Type', and 'Size'. The table is currently empty, with the text 'Working on it...' displayed below the headers.

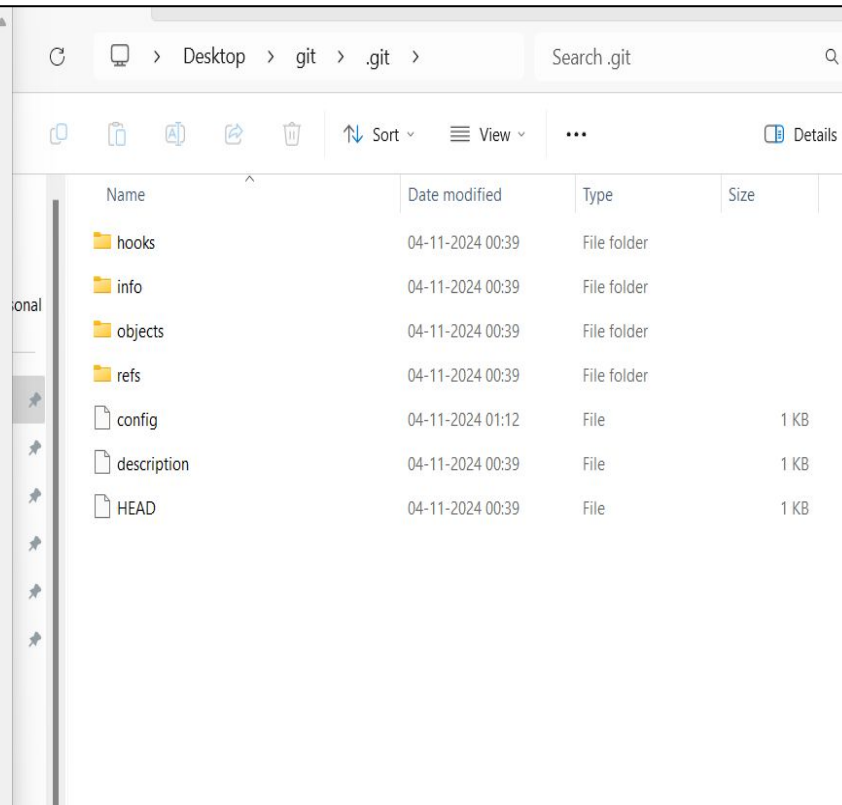
Initialized a Repository

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git init
Reinitialized existing Git repository in C:/Users/kechiya/Desktop/git/.git/
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ ls -la
total 12
drwxr-xr-x 1 kechiya 197121 0 Nov  4 01:08 ./
drwxr-xr-x 1 kechiya 197121 0 Nov  4 00:06 ../
drwxr-xr-x 1 kechiya 197121 0 Nov  4 01:12 .git/
drwxr-xr-x 1 kechiya 197121 0 Nov  4 01:08 webapp/
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ ls .git
HEAD  config  description  hooks/  info/  objects/  refs/
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$
```



Created file,checked the status,add file and committed

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ vi myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git add myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git status
On branch master

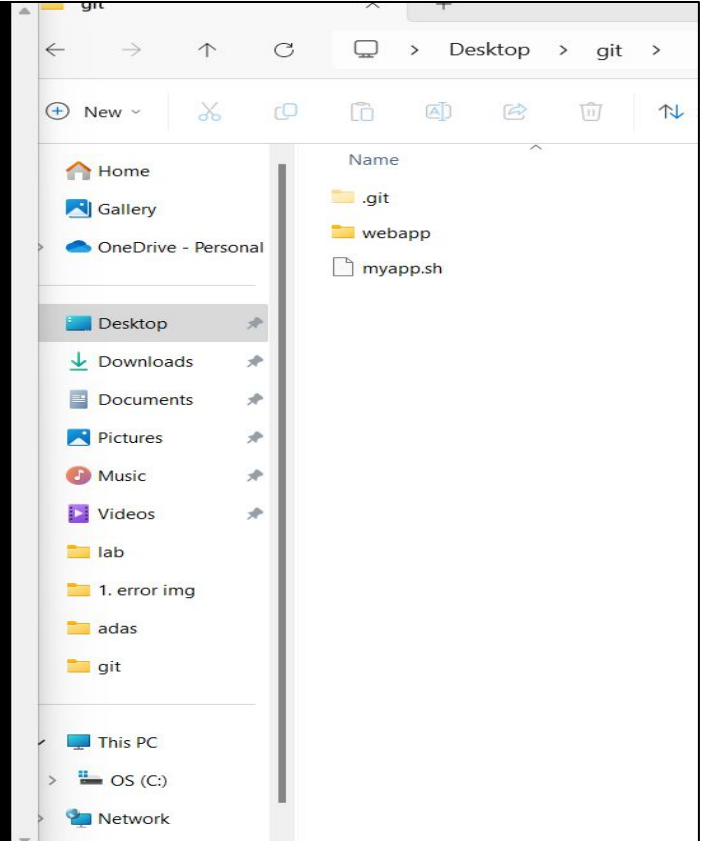
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git commit -m "this is my first line"
[master (root-commit) 5011a99] this is my first line
1 file changed, 2 insertions(+)
create mode 100644 myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git status
On branch master
nothing to commit, working tree clean

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$
```



Two commits in git log

```
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   myapp.sh
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git commit -m "this is my second commit"
[master 57be4f9] this is my second commit
1 file changed, 1 insertion(+)
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git log
commit 57be4f97fc50ac935d1bcf86d690836caac2d20b (HEAD -> master)
```

```
Author: kechiyasunil <kechiya8493@gmail.com>
Date: Mon Nov 4 02:11:01 2024 +0530
```

```
    this is my second commit
```

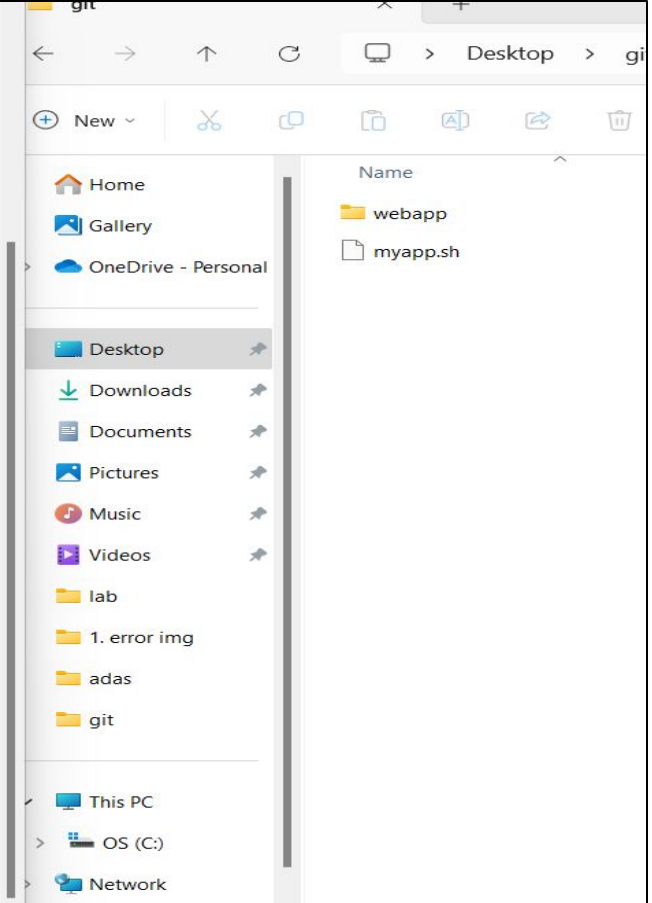
```
commit 5011a9940e800cd1581a12542476742312246245
```

```
Author: kechiyasunil <kechiya8493@gmail.com>
Date: Mon Nov 4 02:00:24 2024 +0530
```

```
    this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git log --oneline
57be4f9 (HEAD -> master) this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$
```



git reset --hard:

- This command is used to **undo changes** in your Git repository by resetting your working directory, staging area (index), and branch pointer to a specific commit.
- It discards any changes to tracked files in the working directory and staging area, making them match the specified commit.

When to Use:

- To undo uncommitted changes completely.
- To reset your repository to a known state (e.g., a specific commit).

Caution:

Use `git reset --hard` with care, especially if there are important uncommitted changes, as they will be permanently discarded.


```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ cat myapp.sh
first line
second line
third line

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git log --oneline
57be4f9 (HEAD -> master) this is my second commit
5011a99 this is my first line

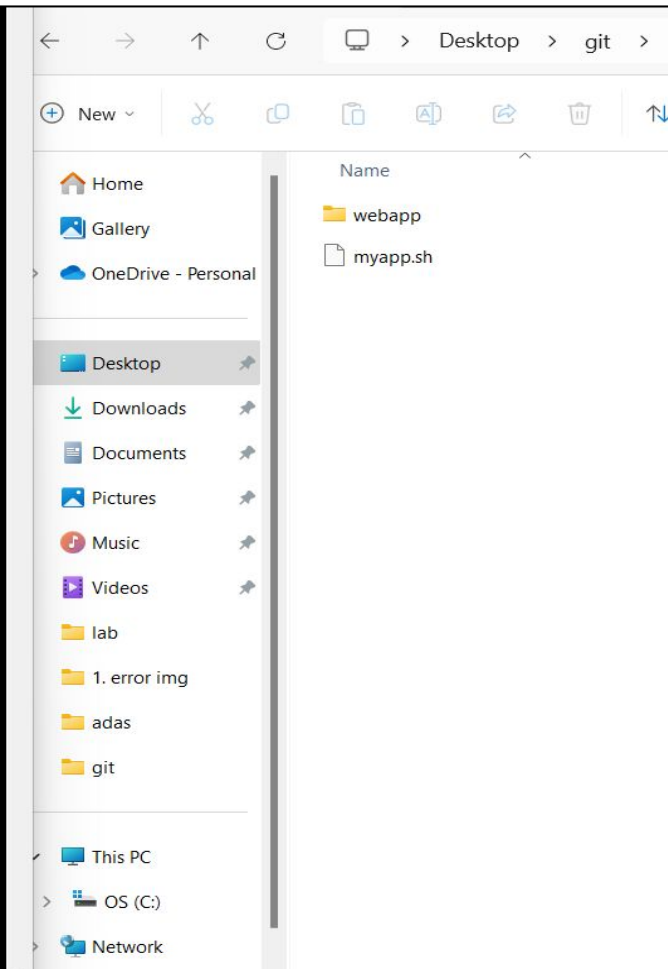
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git reset --hard 5011a99
HEAD is now at 5011a99 this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ cat myapp.sh
first line
second line

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git log
commit 5011a9940e800cd1581a12542476742312246245 (HEAD -
> master)
Author: kechiyasunil <kechiya8493@gmail.com>
Date: Mon Nov 4 02:00:24 2024 +0530

    this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$
```



GitHub:

- Github is a **web-based platform for hosting and collaborating on Git repositories**. It provides tools for version control, project management, and collaboration, all centered around Git.
- GitHub is widely used by developers, teams, and organizations to manage and share code.

1. **Git Repository Hosting:** Hosts Git repositories, allowing developers to store, share, and collaborate on code.
2. **Collaboration Tools:** Features like **pull requests**, **issues**, and **discussions** help teams work together efficiently.
3. **Version Control:** Tracks changes to code, enabling developers to manage different versions and collaborate without overwriting each other's work.
4. **Branching and Merging:** Developers can create branches for new features or bug fixes and later merge them into the main codebase.
5. **Integration and Automation:** Supports integrations with CI/CD tools (e.g., GitHub Actions), enabling automated testing, deployment, and workflows.
6. **Code Review:** Pull request reviews allow teams to discuss, review, and approve changes before merging.
7. **Project Management:** Tools like project boards, milestones, and issues help track progress and organize work.
8. **Open Source Community:** Hosts millions of open-source projects, making it a hub for learning, contributing, and collaborating on public repositories.
9. **Security:** Provides features like Dependabot, secret scanning, and security alerts to help secure your code.



kechiyasunil / kechiyarepo

Q Type to search



<> Code Issues Pull requests Actions Projects Wiki Security Insights Settings



kechiyarepo

Public



Unwatch

1



Fork

0



Star

0



main



1 Branch



0 Tags

Q Go to file



Add file

<> Code



kechiyasunil Initial commit

895ce8d · now

1 Commit



README.md

Initial commit

now



README



kechiyarepo

About

No description, website, or topics provided.

Readme

Activity

0 stars

1 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Integrate git and github to push file to a repository

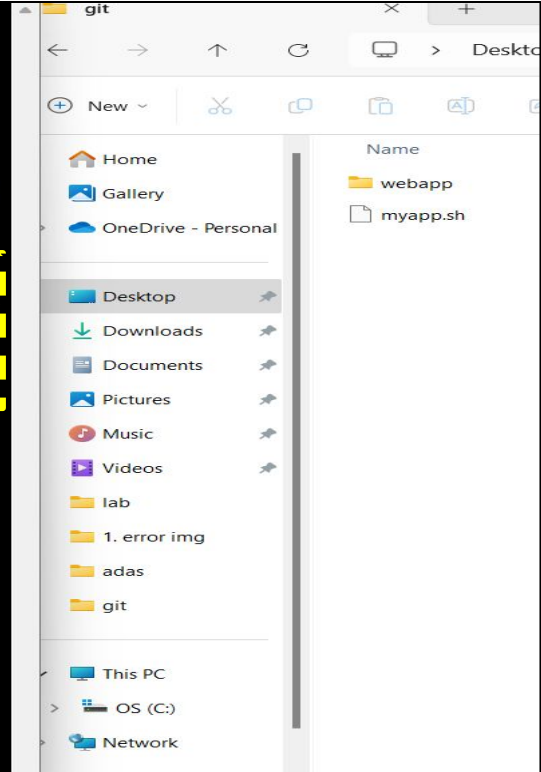
```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git config --global user.name "kechiyasunil"

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git remote -v

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git remote add origin https://github.com/kechiyasunil/kechiyarepo.git

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git remote -v
origin https://github.com/kechiyasunil/kechiyarepo.git (fetch)
origin https://github.com/kechiyasunil/kechiyarepo.git (push)

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$
```



After pushed to github

 kechiyasunil / kechiyarepo

Q Type 7 to search

+ ▾

⌕

⌕

<> Code

⌚ Issues

🔗 Pull requests

⌚ Actions

📁 Projects

📖 Wiki

🛡 Security

📊 Insights

⚙ Settings

Commits

 master ▾

 All users ▾

 All time ▾

 Commits on Nov 17, 2024

4th commit
 kechiyasunil committed 7 minutes ago 08b387a  <>

 Commits on Nov 4, 2024

this is my second commit
 kechiyasunil committed 2 weeks ago 57be4f9  <>

this is my first line
 kechiyasunil committed 2 weeks ago 5011a99  <>

Created dev branch

A **branch** in Git is essentially a **pointer to a specific commit**, allowing you to work on different versions of a project simultaneously. Branches enable developers to work on features, fixes, or experiments independently without affecting the main codebase.

```
MINGW64:/c:/Users/kechiya/Desktop/git
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git branch
* master
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git branch dev
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git branch
* dev
  master
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git switch dev
Switched to branch 'dev'
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git branch
* dev
  master
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ ls
myapp.sh  webapp/
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$
```

In dev branch edited the file and committed it

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ vi myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git add myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git status
On branch dev
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git commit -m "this is my fifth commit"
[dev a2cd759] this is my fifth commit
1 file changed, 1 insertion(+), 1 deletion(-)

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git status
On branch dev
nothing to commit, working tree clean

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git log --oneline
a2cd759 (HEAD -> dev) this is my fifth commit
08b387a (origin/master, master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git log --oneline
a2cd759 (HEAD -> dev) this is my fifth commit
08b387a (origin/master, master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git switch master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ cat myapp.sh
first line
third line

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git switch dev
Switched to branch 'dev'

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ cat myapp.sh
first line
third line
fourth line

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ |
```


I created test branch from dev,so it reflected dev branch commits in test

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git checkout -b test
Switched to a new branch 'test'

kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ vi myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git add myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git commit -m "this is my first commit"
[test 508539d] this is my first commit
1 file changed, 2 insertions(+)

kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git status
On branch test
nothing to commit, working tree clean

kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git log --online
fatal: unrecognized argument: --online

kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git log --oneline
508539d (HEAD -> test) this is my first commit
a2cd759 (dev) this is my fifth commit
08b387a (origin/master, master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (test)
```

It displays all of my commits across all 3 branches

```
kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git log --oneline
508539d (HEAD -> test) this is my first commit
a2cd759 (dev) this is my fifth commit
08b387a (origin/master, master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git switch dev
Switched to branch 'dev'
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git log --oneline
a2cd759 (HEAD -> dev) this is my fifth commit
08b387a (origin/master, master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git switch master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git log --oneline
08b387a (HEAD -> master, origin/master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
```

The **git cherry-pick** command is used to apply a specific commit (or commits) from one branch into another branch. This is particularly useful when you want to pick a particular change without merging an entire branch.

```
Switched to branch 'dev'

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git log --oneline
a2cd759 (HEAD -> dev) this is my fifth commit
08b387a (origin/master, master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git checkout master
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ cat myapp.sh
first line
third line

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git cherry-pick a2cd759
[master 2b78f81] this is my fifth commit
Date: Sun Nov 17 21:45:43 2024 +0530
1 file changed, 1 insertion(+), 1 deletion(-)

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ cat myapp.sh
first line
third line
fourth line

kechiya@Sunil MINGW64 ~/Desktop/git (master)
```

Now we can see master and dev have same commits

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git log --oneline
2b78f81 (HEAD -> master) this is my fifth commit
08b387a (origin/master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git checkout dev
Switched to branch 'dev'
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git log --oneline
a2cd759 (HEAD -> dev) this is my fifth commit
08b387a (origin/master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$
```


Creating another branch (prod)

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git branch prod

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git switch prod
Switched to branch 'prod'

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ vi myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ git commit -am "this is my first commit "
[prod Odd6649] this is my first commit
1 file changed, 2 insertions(+)

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ git status
On branch prod
nothing to commit, working tree clean

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ vi myapp.sh

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ git commit -am "this is my second commit in prod"
[prod 6419108] this is my second commit in prod
1 file changed, 1 insertion(+), 1 deletion(-)

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ git status
On branch prod
nothing to commit, working tree clean
```

Rebase :

- Rebase in Git is an operation that **reapplies commits from one branch into another base**, effectively changing the commit history.
- It's often used to create a linear, cleaner project history by integrating changes from one branch into another without creating a merge commit.

```
kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git rebase prod
Successfully rebased and updated refs/heads/dev.

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git log --oneline
6419108 (HEAD -> dev, prod) this is my second commit in prod
0dd6649 this is my first commit
a2cd759 this is my fifth commit
08b387a (origin/master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (dev)
$ git switch prod
Switched to branch 'prod'

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ git log --oneline
6419108 (HEAD -> prod, dev) this is my second commit in prod
0dd6649 this is my first commit
a2cd759 this is my fifth commit
08b387a (origin/master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$
```

Merge is the process of integrating changes from one branch into another. It combines the histories of two branches, enabling you to bring changes together and continue working on a unified codebase.

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git merge test
Already up to date.
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git log --oneline
6c95036 (HEAD -> master) Merge branch 'test'
b1ce535 this is my first commit
2b78f81 this is my fifth commit
508539d (test) this is my first commit
a2cd759 this is my fifth commit
08b387a (origin/master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git switch test
Switched to branch 'test'
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (test)
$ git log --oneline
508539d (HEAD -> test) this is my first commit
a2cd759 this is my fifth commit
08b387a (origin/master) 4th commit
57be4f9 this is my second commit
5011a99 this is my first line
```

```
kechiya@Sunil MINGW64 ~/Desktop/git (test)
$
```


Pushed all the branches into github

```
kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git push --set-upstream origin master
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (9/9), 792 bytes | 396.00 KiB/s, done.
Total 9 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/kechiyasunil/kechiyarepo.git
    08b387a..6c95036  master -> master
branch 'master' set up to track 'origin/master'.

kechiya@Sunil MINGW64 ~/Desktop/git (master)
$ git checkout prod
Switched to branch 'prod'

kechiya@Sunil MINGW64 ~/Desktop/git (prod)
$ git push --set-upstream origin prod
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (6/6), 538 bytes | 269.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'prod' on GitHub by visiting:
remote:   https://github.com/kechiyasunil/kechiyarepo/pull/new/prod
remote:
```

 kechiyarepo

Public

 Pin Unwatch **prod** had recent pushes 8 minutes ago

Compare & pull request

 **dev** had recent pushes 3 minutes ago

Compare & pull request

 **test** had recent pushes 2 minutes ago

Compare & pull request

 master 5 Branches 0 Tags

Go to file

t

Add file

<> Code

This branch is 8 commits ahead of, 1 commit behind main .

 Contribute kechiyasunil Merge branch 'test'

6c95036 · 23 minutes ago

 8 Commits myapp.sh

this is my first commit

24 minutes ago

 README

Branches

[New branch](#)[Overview](#)[Yours](#)[Active](#)[Stale](#)[All](#)

Default

Branch	Updated	Check status	Behind	Ahead	Pull request
main	4 hours ago			Default	...

Your branches

Branch	Updated	Check status	Behind	Ahead	Pull request
test	4 minutes ago		1	5	...
dev	5 minutes ago		1	6	...
prod	9 minutes ago		1	6	...
master	13 minutes ago		1	8	...

Active branches

Branch	Updated	Check status	Behind	Ahead	Pull request
test	4 minutes ago		1	5	...
dev	5 minutes ago		1	6	...
prod	9 minutes ago		1	6	...
master	13 minutes ago		1	8	...